

Homework Project — From Notebook to Pipeline

Welcome to the real world of Data Engineering! 🛠️

0. Where you are

In the lab, you built the RICO multimodal pipeline by hand, top-to-bottom, in a single notebook: 5 screens from HuggingFace into MinIO, parse the view hierarchy, embed images with CLIP, embed text with SBERT, extract structured JSON with a local LLM, and search across the triad. You also saw — on purpose — what happens on the second run: primary-key violations, no idempotency, no recovery, no observability.

The notebook crashed beautifully, and the lesson landed!

You are now responsible for taking that same conceptual pipeline and turning it into something a team would actually run in production.

1. The project, in one sentence

Re-implement the lab notebook's pipeline as an Airflow DAG, with row-level traceability, a duplicate-detection audit that halts the pipeline on failure, and observability metrics that report the health of every run and the quality of the data landing in the destination tables.

Same data. Same models. Same three stores (MinIO, Postgres+pgvector, the LLM endpoint). Different shape: a scheduled, idempotent, observable, auditable DAG instead of a notebook that runs once and dies.

2. What you have, what you don't

You have: - The lab notebook (`lab/notebook.ipynb`) — your conceptual reference. Every primitive you need is in there. - Running infrastructure (MinIO, Postgres+pgvector, Ollama) from the same `make up` you used in the lab. - The same dataset (`rootsautomation/RICO-Screen2Words` on HuggingFace).

You do not have: - The production codebase. You build this from scratch. - Test fixtures, helper modules, prompts files, or wiring code from the production project. If you want a versioned prompt, you write one yourself. - Permission to copy from `src/`. The notebook is your only allowed reference for code shape; everything you write is yours.

You're starting from a blank `pyproject.toml` and an empty `dags/` folder.

3. Required deliverables

3.1 The DAG (the obvious part)

A single Airflow DAG that runs the seven stages from the lab, in this order:

```
ingest → parse → [embed_image, embed_text, extract] → load → audit → eval
```

Notes on shape: - The DAG file is thin. Orchestration only — no business logic in the DAG. - The three middle tasks (`embed_image` , `embed_text` , `extract`) run in parallel. - A `LIMIT` parameter controls how many screens to process, so you can dev on 5 and demo on 50. - Re-running the DAG with the same `LIMIT` must not create duplicate rows or duplicate blobs. **Idempotency is now your problem.** - The stages must read and write to the same Postgres tables and MinIO bucket your lab notebook used.

This is the part that mirrors the lab. Do it cleanly, but it is not where you earn the bulk of your grade. The next four sections are.

3.2 Record traceability

Every row in every destination table must answer the question: "Where did this come from, and which run produced it?"

Concretely: - Introduce a `pipeline_runs` table (or equivalent) with at minimum: `run_id` (UUID), `dag_run_id` (Airflow's), `started_at` , `ended_at` , `status` , `limit_param` , `git_sha` (the commit that the code is running from), and the model versions used in that run (CLIP version, SBERT version, LLM model + prompt version). - Every existing destination table (`screens_metadata` , `screens_embeddings` , `screens_review_queue`) gains a `run_id` foreign key to `pipeline_runs` . A row is now traceable to the exact run that wrote it. - Every row also gains a `source_fingerprint` — a hash (SHA-256 is fine) of the input that produced it. For ingested screens, hash the PNG bytes. For embeddings, hash the input that fed the embedder. This lets you answer "did the model see exactly this byte sequence?" without storing the bytes in Postgres. - Logs include the `run_id` on every line. A failed task should be diagnosable from logs alone — given a `run_id` , you can find every row, every blob, every log line.

Why this matters: In production, when retrieval quality drops, the first question is "what changed?" Without traceability you cannot answer it. With traceability you can.

3.3 The audit (one is enough — make it real)

After the load stage and before eval, the DAG runs a single audit task. You must pick the duplicate-detection audit — that is the one this assignment requires.

Concretely: - The audit checks that no (`screen_id`, `model_name`, `model_version`, `embedding_kind`) combination appears more than once in `screens_embeddings`, and that no `screen_id` appears more than once in `screens_metadata` for the current run. - If the audit fails, the DAG pauses — i.e., the audit task fails loudly, downstream tasks (eval) do not run, and the run is marked failed. The bad data is not propagated. - The audit logs the duplicate keys it found, in full, so a human can investigate. - The audit is its own task, not a side effect of load. It must be visible as a node in the DAG graph. - *Bonus:* Store the audit result (`run_id`, `audit_name`, `passed`, `details`) in an `audit_results` table so audit history is queryable. Not required, but encouraged.

Read the audit as a circuit breaker: It has the authority to stop the pipeline. That authority is what makes it useful. A "warning" that nobody acts on is not an audit.

3.4 Observability — *health + data quality*

You will report on two things at the end of every run.

Pipeline health metrics: - Per-task duration (seconds). - Per-task row count in / row count out (where applicable). - Total run duration. - Number of retries per task. - Final run status (succeeded / failed / paused-by-audit).

Data quality metrics on destination tables (post-load): - `screens_metadata`: row count for the run, % of rows with non-null `extraction_payload`, % of rows with `confidence` ≥ 0.5 , % of rows in `screens_review_queue`. - `screens_embeddings`: row count per (`model_version`, `embedding_kind`), average vector dimensionality (it should be a constant — flag if it's not), % of rows whose vector norm is exactly zero (pure-zero vectors are a silent embedder bug). - Distinct count of `app_package` and `category` (sanity check: did we accidentally process the same app over and over?).

Where these go: - Persist them in a `pipeline_metrics` table keyed by `run_id` and `metric_name`, so you can plot them across runs later. - At the end of every run, log a one-screen-tall summary of the metrics so an operator skimming Airflow logs sees the health of the run without leaving the UI. - *Optional stretch:* Expose them on a `/metrics` HTTP endpoint or push them to a Prometheus-style scraper. Not required; the table + log line is enough.

Why this matters: A pipeline you cannot observe is a pipeline you cannot trust. The metrics are the evidence — when somebody asks "is the pipeline healthy?", you do not answer "I think so", you query the metrics table.

3.5 Slack notifications

Every run posts to a Slack channel (your team's, or a dedicated `#pipeline-alerts`) at three moments: - **Run started** — `run_id`, `LIMIT`, what triggered it (manual / scheduled). - **Audit failed** — the duplicate keys the audit found, the `run_id`, and a link back to the Airflow task log. - **Run finished** — final status

(succeeded / failed / paused-by-audit), total duration, and the one-line health + data-quality summary from §3.4.

Implementation notes: - Use an incoming-webhook URL stored in an Airflow Connection or an environment variable. **Never commit the URL to git.** - A failure to post to Slack must not fail the run. Notifications are observability, not a pipeline dependency — wrap the post in a `try/except`, log the failure, and move on. - The audit-failed message is the most important one; make it scannable. An on-call engineer should know from the Slack message alone whether to wake up or wait until morning.

4. What you are not building

To keep the scope honest: - No frontend. The artifact is the DAG, the populated tables, and the metrics. Validation is by SQL. - No model retraining. Use the same models as the lab. - No new modalities. No audio, no video. - **No alerting integrations beyond Slack.** Slack notifications are required (§3.5); PagerDuty, email, or anything else is out of scope. - You may keep the eval simple — `recall@5` with a self-test holdout is acceptable; you already saw why it's a tautology. *Optional stretch:* Implement a disjoint holdout the way Section 7 of the lab demonstrated.

5. Definition of Done

- `make up` brings up infrastructure; your DAG appears in the Airflow UI without errors.
- Triggering the DAG with `LIMIT=5` populates all destination tables and produces a `pipeline_runs` row + a `pipeline_metrics` row.
- Re-triggering the DAG with `LIMIT=5` produces no new rows in any destination table (idempotent).
- Manually corrupting `screens_embeddings` (e.g., insert a duplicate (`screen_id`, `model_version`, `embedding_kind`) row) and re-running causes the audit task to fail and eval to be skipped.
- Every row in every destination table has a non-null `run_id` and `source_fingerprint`.
- The end-of-run log line shows the health + data quality summary, readable in 10 seconds.
- A successful run, a failed run, and an audit-halted run each post the expected message to Slack; the webhook URL is not committed to the repo.
- A README explains how to run the DAG, what each metric means, and how to interpret an audit failure.

6. How you will be evaluated

Roughly: - **40% — Correctness and idempotency.** Does the pipeline actually work, and does re-running it leave the world in the same state? - **20% — Traceability.** Can you answer "which run produced this row, with which model, from which source bytes?" using only SQL? - **20% — Audit.** Is the audit a real circuit

breaker (fails loudly, stops the run, logs detail)? - 15% — **Observability**. Are the metrics honest, persisted, and readable? - 5% — **Code quality and README**. Clean separation between DAG and logic; README a stranger can follow.

7. Tips from someone who has done this before

- **Start with traceability schema, not code.** Sketch the new tables (`pipeline_runs` , `audit_results` , `pipeline_metrics`) and the `run_id` columns on existing tables before you write your first task. Once the schema is honest, the DAG writes itself.
- **Make the audit fail at least once on purpose.** Insert a fake duplicate, run the DAG, watch the audit halt the pipeline. If you have never seen your own circuit breaker fire, it does not work.
- **Re-read your lab notebook before you write the embed task.** All the primitives (CLIP loading, MinIO PUT/GET, raw SQL) are in there. You are translating, not redesigning.
- **The DAG file should be boring.** If your DAG file has business logic, you have put logic in the wrong place. Tasks call functions; functions live in your own modules.
- **Idempotency is mostly `INSERT ... ON CONFLICT`.** The lab refused to use it on purpose, to teach you the failure mode. Use it now.

Closing thought. The lab gave you the *what*. This homework gives you the *how it survives in production*. The pipeline that crashes on its second run is a learning artifact. The pipeline that runs every night, traces every row, audits its own outputs, alerts your team on Slack, and reports its health — *that* is the engineering job. Good luck!